


# Big Data Programming

## Basic Operators and Control Statement

# Quick Review

## 계산한 조건을 실행 흐름에 연결하기

### 입력한 값을 조건 판단에 사용한다

input()은 문자열을 반환하므로 숫자를 비교하려면 필요한 변환을 먼저 한다.
int(input(...))으로 받은 정수는 산술 연산과 숫자 비교에 사용할 수 있다.

### 대입과 비교를 구분한다

score = 60은 값을 대입하고 score >= 60은 현재 값에 대한 조건을 계산한다. 조건식을 적었다는 사실만으로 특정 문장이 선택되어 실행되지는 않는다.

### 오늘은 “무엇을 계산하는가”에서 한 단계 더 나아간다

if는 실행할 문장을 선택하고 while과 for는 실행을 반복한다. 각 문장에서 어떤 값을 사용하는지, 다음에 어느 줄로 이동하는지를 함께 읽는다.

# Control Statement

## 순차·선택·반복 구조로 흐름 구분하기

### 순차 구조는 정해진 순서대로 실행한다

입력받고 계산한 뒤 출력하는 코드는 위에서 아래로 실행된다. 앞 문장의 실행 결과가 뒤 문장에서 사용하는 값에 영향을 준다.

### 선택 구조는 조건에 따라 경로를 바꾼다

if, if/else, if/elif/else를 사용한다. 조건이 참인지 거짓인지에 따라 실행되는 본문이 달라지므로 모든 줄이 항상 실행되는 것은 아니다.

### 반복 구조는 같은 본문을 다시 실행한다

while은 조건을 다시 검사하고, for는 다음 원소를 가져오며 반복한다. 실행 횟수뿐 아니라 매 반복에서 변수의 값이 어떻게 달라지는지도 확인한다.

### 구조별 질문

| 구조 | 코드를 읽을 때 묻는 질문 |
|---|---|
| 순차 | 다음에 실행할 문장은 무엇인가? |
| 선택 | 이 조건에서 어떤 본문을 실행하는가? |
| 반복 | 다시 실행하는 이유와 끝나는 조건은 무엇인가? |

### 함께 사용하는 구조

반복문 안에서 조건문을 사용할 수도 있다. 복잡한 코드도 기본 구조를 나누어 보면 읽을 수 있다.

# Sequential Structure

## 앞에서 만든 값을 다음 문장에서 사용한다

### 예제의 초기 상태를 먼저 정한다

total은 누적 점수, counter는 처리한 점수의 개수이다. 여기서는 total = 150, counter = 2, grade = 80을 초기값으로 두고 두 문장의 실행을 살펴본다.

### 대입문의 오른쪽은 갱신 전 값을 사용한다

total = total + grade는 150 + 80을 먼저 계산한 뒤 total을 230으로 갱신한다. 이어서 counter = counter + 1은 2 + 1을 계산해 3을 저장한다.

### 순차 실행에도 상태 추적이 필요하다

같은 이름이 문장의 양쪽에 있어도 모순된 등식이 아니다. 코드에 값을 적지 않고 total이나 counter를 먼저 사용하면 NameError가 발생할 수 있다.

### 초기값과 실행할 문장

```
total = 150
counter = 2
grade = 80
total = total + grade
counter = counter + 1
print(total, counter)
```

### 실행 결과

```
230 3
```

### 같은 정수 예제의 짧은 표현

total += grade와 counter += 1로도 갱신할 수 있다. 우선 긴 형태로 실행 순서를 이해한다.

# Code Blocks

## 콜론과 들여쓰기가 본문의 범위를 정한다

### 콜론 뒤에 실행할 본문이 이어진다

if와 while, for의 머리 부분은 콜론으로 끝난다. 그 아래에서 들여쓴 문장들이 해당 구조의 본문이다. 중괄호 대신 들여쓰기로 묶음을 나타낸다.

### 같은 본문의 문장은 같은 깊이로 맞춘다

이 수업에서는 공백 4칸을 한 단계로 사용한다. 탭과 공백을 섞지 않고, VS Code의 들여쓰기 설정을 일관되게 사용한다.

### 들여쓰기가 끝나면 본문 밖으로 나온다

오른쪽 예제의 첫 두 print()는 조건이 참일 때만 실행된다. 마지막 print()는 if 본문 밖에 있으므로 조건이 거짓이어도 실행된다.

### 본문의 범위를 확인

```
score = 70
if score >= 60:
    print("Passed")
    print("조건을 만족합니다.")
print("검사를 마쳤습니다.")
```

### score가 50이면

들여쓴 두 출력문은 건너뛴다. “검사를 마쳤습니다.”만 출력된다.

### 오류와 의미 변화를 구분

잘못된 들여쓰기는 오류를 만들 수도 있고, 문법은 맞지만 의도와 다른 실행 범위를 만들 수도 있다.

# Selection Structure

## if: 조건이 참일 때만 본문 실행하기

### 조건식을 먼저 평가한다

if score >= 60:에서는 현재 score의 값을 읽어 비교한다. 조건이 참이면 들여쓴 본문으로 들어가고, 거짓이면 본문을 건너뛴다.

### 거짓인 경우에도 다음 코드로 진행한다

단독 if에는 거짓일 때 실행할 별도 본문이 없다. 그렇다고 전체 프로그램이 끝나는 것은 아니다. if 다음에 있는 문장으로 실행을 계속한다.

### 예제의 기준과 실제 규정을 구분한다

이 예제의 60점은 조건문을 배우기 위한 판정 기준이다. 실제 과목의 합격이나 성적 부여 기준을 새로 정하는 내용은 아니다.

### 기본 형태

```
if condition:
    statement
```

### 점수 판정 예제

```
score = 70
if score >= 60:
    print("Passed")
print("Done")
```

### score가 70일 때

```
Passed
Done
```

# Selection Structure

## if 실행 경로를 입력값별로 추적하기

score를 70으로 두면 비교 결과가 True이고 50으로 두면 False이다. “Passed”를 출력하는 문장을 실행하는지는 이 결과에 의해 결정된다.

| score | score >= 60 | if 본문 | 본문 밖의 print("Done") |
|---|---|---|---|
| 70 | True | 실행한다 | 실행한다 |
| 60 | True | 실행한다 | 실행한다 |
| 50 | False | 건너뛴다 | 실행한다 |

### 실행되지 않은 줄을 구분한다

score가 50일 때 “Passed”가 출력되지 않는 것은 프로그램이 중간에 멈췄기 때문이 아니다. 조건을 계산한 결과 해당 줄을 선택하지 않았기 때문이다.

### 확인 질문

score > 60으로 조건을 바꾸면 표의 어느 행이 달라지는가?
60점 행만 달라지며, 나머지 두 입력에서는 같은 경로가 선택된다.

# Selection Structure

## if/else: 두 경로 중 하나 선택하기

### 조건이 참이면 if 본문을 실행한다

score >= 60이 참이면 “Passed”를 출력한다. 이 경우 else 본문은 실행하지 않으며 선택 구조 뒤의 문장으로 이동한다.

### 조건이 거짓이면 else 본문을 실행한다

else는 앞의 if 조건이 거짓인 경우를 담당한다. else 뒤에 새로운 조건식을 쓰지 않고 콜론을 붙인다. 두 본문은 같은 if/else 구조에 속한다.

### 하나를 실행한 뒤 공통 흐름으로 합쳐진다

정상적인 실행에서는 두 경로 중 하나만 선택된다. 마지막의 “Done”은 두 경로가 끝난 뒤 공통으로 실행되는 문장이다.

### 예제 코드

```
score = 50
if score >= 60:
    print("Passed")
else:
    print("Failed")
print("Done")
```

### score가 50일 때

```
Failed
Done
```

### 들여쓰기 확인

if와 else는 같은 깊이이다. 각 본문은 그보다 한 단계 들여쓴다.

# Selection Structure

## 경계값을 포함하는지 확인하기

### 이상과 초과는 다른 조건이다

60점 이상은 score >= 60이고 60점 초과는 score > 60이다. 조건이 비슷해 보이더라도 경계값에서 선택되는 경로가 달라진다.

### 경계 주변의 세 값을 넣어 본다

기준보다 작은 59, 기준과 같은 60, 기준보다 큰 61을 사용한다. 대표적인 값 하나만 시험하는 것보다 잘못된 비교 기호를 찾기 쉽다.

### 입력 범위를 따로 정해야 할 수도 있다

예제가 0점부터 100점까지를 가정한다면 -10이나 150은 별도 검사가 필요한 입력이다. 문법적으로 비교할 수 있는 값과 문제에서 허용하는 값은 다르다.

### 두 조건 비교

| score | >= 60 | > 60 |
|---|---|---|
| 59 | False | False |
| 60 | True | False |
| 61 | True | True |

### 범위 자체를 확인하는 식

0 <= score <= 100

### 연결 비교

위 식은 score가 0 이상이고 100 이하인지 확인한다. 두 경계가 모두 포함된다.

# Selection Structure

## if/elif/else: 여러 조건을 순서대로 검사하기

### 위에서 아래로 조건을 검사한다

if 조건이 거짓이면 다음 elif 조건을 검사한다. 처음 참이 된 조건의 본문을 실행한 뒤에는 이 선택 구조의 나머지 조건을 더 검사하지 않는다.

### 어느 조건도 참이 아니면 else로 간다

else는 앞의 모든 조건이 거짓인 경우를 담당한다. Elif는 여러 개 사용할 수 있고, 기본 경로가 필요하지 않다면 else를 생략할 수도 있다.

### 한 번의 분류인지 여러 번의 검사인지 정한다

아래 코드는 실습용 점수 분류이다. 한 입력에 하나의 분류를 출력하려는 목적이므로 여러 조건을 하나의 if/elif/else로 연결한다.

### 실습용 분류 예제

```
score = 85
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("C")
```

### 실행 결과

B

### 85가 검사되는 순서

85 >= 90은 거짓이다.
85 >= 80은 참이므로 B를 출력한다.

# Selection Structure

## 여러 if와 if/elif는 같은 구조가 아니다

### 독립된 if는 각각 조건을 검사한다

score가 95이면 score >= 90과 score >= 80은 모두 참이다. 두 개의 독립된 if를 사용하면 두 본문이 모두 실행된다.

### elif는 앞 조건이 거짓일 때만 검사한다

if/elif 구조에서는 첫 조건이 참이면 해당 본문을 실행하고 구조를 빠져나온다. 따라서 score가 95일 때 A만 출력된다.

### 원하는 동작으로 구조를 선택한다

여러 사실을 모두 확인하려면 독립된 if가 필요할 수 있다. 하나의 등급이나 범주만 선택하려면 if/elif가 더 자연스럽다.

### 두 조건을 각각 검사

```
score = 95
if score >= 90:
    print("A")
if score >= 80:
    print("B")
```

A와 B가 각각 출력된다.

### 한 경로만 선택

```
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
```

A만 출력된다.

# Selection Structure

## 조건을 나열하는 순서도 결과를 바꾼다

### 넓은 조건이 먼저 있으면 뒤 조건에 도달하지 않는다

score >= 60을 먼저 검사하면 95도 그 조건을 만족한다. 같은 구조의 뒤쪽에 score >= 90을 적어도 95의 분류에는 사용되지 않는다.

### 의도한 분류 순서로 조건을 배치한다

높은 점수부터 구간을 나누려면 높은 기준을 먼저 둔다. 앞 조건이 거짓이었다는 사실이 다음 elif에서 가능한 값의 범위를 좁혀 준다.

### 숫자를 넣어 경로를 확인한다

95, 85, 65처럼 서로 다른 구간의 값을 넣어 실제로 선택되는 본문을 확인한다. 조건 문장의 뜻이 맞아도 배치 순서가 틀리면 분류는 잘못될 수 있다.

### 의도와 다르게 분류하는 예

```
score = 95
if score >= 60:
    print("60점 이상")
elif score >= 90:
    print("90점 이상")
```

### 실행 결과

```
60점 이상
```

### 높은 기준을 먼저 검사

```
if score >= 90:
    print("90점 이상")
elif score >= 60:
    print("60점 이상")
```

# Leap Year

## 윤년의 규칙을 세 조건으로 나누기

### 기본 조건은 4로 나누어떨어지는가이다

year % 4 == 0은 해당 연도가 4의 배수인지 확인한다. 하지만 이 조건만으로는 100의 배수인 해를 모두 올바르게 판단할 수 없다.

### 100의 배수는 제외하되 400의 배수는 포함한다

규칙은 “4의 배수이면서 100의 배수가 아닌 해, 또는 400의 배수인 해”로 묶을 수 있다. 예외 조건까지 포함해야 전체 규칙이 된다.

### 조건의 관계를 자연어로 먼저 읽는다

and는 기본 조건과 제외 조건을 함께 적용한다. Or는 400의 배수라는 별도의 포함 조건을 연결한다. 괄호를 사용하면 이 관계가 더 명확해진다.

### 세 개의 조건

year % 4 == 0
year % 100 != 0
year % 400 == 0

### 예상할 사례

2012는 윤년이다.
1900은 윤년이 아니다.
2000은 윤년이다.

### 규칙의 적용 범위

이 실습에서는 그레고리력의 윤년 규칙을 적용할 연도를 입력한다고 가정한다.

# Practice

## 실습: 입력된 연도가 윤년인지 판별하기

### 입력한 연도를 정수로 변환한다

year에 정수 연도를 저장한다. 나머지 연산으로 나누어떨어지는지 검사해야 하므로 문자열을 그대로 비교하지 않는다.

### 조건식을 완성한 뒤 두 경로를 만든다

윤년 조건을 만족하면 “윤년입니다.”를, 그렇지 않으면 “윤년이 아닙니다.”를 출력한다. 출력에 사용한 연도와 판단한 연도가 같은지 확인한다.

### 하나의 성공 사례로 끝내지 않는다

2012뿐 아니라 1900과 2000도 시험한다. 4의 배수 조건만 사용했거나 400의 배수 예외를 빠뜨리면 이 두 입력에서 차이가 나타난다.

### 완성할 코드

```
year = int(input("연도: "))
if _____________________:
    print(year, "년은 윤년입니다.")
else:
    print(year, "년은 윤년이 아닙니다.")
```

### 먼저 적을 예상 결과

2012 → 윤년
1900 → 평년
2000 → 윤년
2023 → 평년

# Practice

## 해설: 윤년 조건을 괄호와 논리 연산으로 연결하기

### 조건식의 두 묶음을 구분한다

첫 묶음은 4의 배수이면서 100의 배수가 아닌 경우이다. 두 번째 묶음은 400의 배수인 경우이다. 둘 중 하나만 만족해도 윤년으로 판단한다.

### 중간 조건에 이름을 붙여 읽을 수 있다

아래 코드는 긴 조건을 is_leap에 먼저 대입한 변형이다. 같은 조건을 if 뒤에 직접 적은 형태와 같은 판별을 수행하지만 각 단계를 읽기 쉽다.

### 1900과 2000의 차이를 설명한다

1900은 첫 묶음도 거짓이고 두 번째 묶음도 거짓이다. 2000은 첫 묶음은 거짓이지만 400의 배수이므로 두 번째 묶음이 참이다.

### 읽기 쉽게 나눈 해설 코드

```
year = int(input("연도: "))
is_leap = (
    (year % 4 == 0 and year % 100 != 0)
    or year % 400 == 0
)
if is_leap:
    print(year, "년은 윤년입니다.")
else:
    print(year, "년은 윤년이 아닙니다.")
```

### 한 문장으로 읽기

4의 배수이되 100의 배수는 아니거나, 400의 배수이면 윤년이다.

# Leap Year

## 입력별 조건의 참과 거짓을 표로 확인하기

A는 year % 4 == 0, B는 year % 100 != 0, C는 year % 400 == 0이다. 최종 조건은 (A and B) or C로 계산한다.

| year | A | B | C | 최종 조건 |
|---|---|---|---|---|
| 2012 | True | True | False | True |
| 1900 | True | False | False | False |
| 2000 | True | False | True | True |
| 2023 | False | True | False | False |

### 잘못된 식과 비교한다

A and (B or C)는 이 규칙에서는 같은 결과를 낸다. 다만 임의의 세 조건이 언제나 이렇게 바뀌는 것은 아니다. 여기서는 400의 배수이면 반드시 4의 배수라는 관계가 있기 때문이다.

### 검증의 목적

조건식의 겉모양을 외우기보다 각 조건의 의미를 설명한다. 일반적으로 괄호를 바꿀 때는 논리 관계와 실제 입력을 모두 확인해야 한다.

# Repetition Structure

## 같은 본문을 여러 번 실행하는 두 방식

### while은 조건을 기준으로 반복한다

조건이 참인 동안 본문을 실행한다. 처음부터 정확한 반복 횟수를 정하기 어렵고, 계산 중 바뀌는 상태로 종료를 판단할 때 사용할 수 있다.

### for는 다음 원소를 하나씩 사용한다

문자열이나 range처럼 순회할 수 있는 대상에서 원소를 하나씩 가져와 본문을 실행한다. range를 사용하면 정해진 정수 범위에 대해 반복할 수 있다.

### 둘 다 본문과 다음 단계의 관계가 중요하다

while에서는 조건을 다시 검사하고 for에서는 다음 원소를 가져온다. 반복문 밖의 문장은 반복이 끝난 뒤 한 번 실행된다.

### 조건을 기준으로 반복

```
while condition:
    statement
```

### 원소를 순서대로 사용

```
for item in iterable:
    statement
```

### 앞으로 확인할 질문

초기값은 무엇인가?
본문에서 무엇이 바뀌는가?
반복은 언제 끝나는가?

# While Repetition

## while: 조건 검사 → 본문 → 다시 조건 검사

### 본문에 들어가기 전에 조건을 검사한다

while product <= 1000:은 현재 product가 1000 이하인지 확인한다. 참이면 본문을 실행하고, 거짓이면 반복문 다음 줄로 이동한다.

### 본문에서 상태를 바꾼다

product = 2 * product는 현재 값을 두 배로 만든다. 다시 조건을 검사할 때에는 갱신된 값을 사용하므로 반복할수록 종료 조건에 가까워진다.

### 초기값을 정해야 실행 과정을 알 수 있다

여기서는 product를 2로 시작한다. 반복문 자체가 시작값을 정해 주는 것은 아니다. 조건에 쓰는 이름을 먼저 정의하고 어떤 값으로 출발하는지 확인한다.

### 초기값을 둔 예제

```
product = 2
while product <= 1000:
    product = 2 * product
print(product)
```

### 실행 결과

1024

### 1000보다 큰 결과가 나오는 이유

조건은 곱셈 전에 검사한다. 512에서 본문에 들어간 뒤 1024가 되고, 다음 검사에서 반복이 끝난다.

# While Repetition

## 반복 직전과 직후의 값을 추적하기

초기 product는 2이다. 아래 표의 반복 번호는 본문 실행 횟수이며, 각 행은 한 번의 조건 검사와 본문 실행을 보여 준다.

| 본문 실행 횟수 | 검사 시 product | <= 1000 | 본문 실행 후 |
|---|---|---|---|
| 1 | 2 | True | 4 |
| 2 | 4 | True | 8 |
| 3 | 8 | True | 16 |
| … | … | … | … |
| 8 | 256 | True | 512 |
| 9 | 512 | True | 1024 |
| 다음 검사 | 1024 | False | 본문을 실행하지 않음 |

### 종료 시점의 상태를 설명한다

본문은 총 9번 실행되고 마지막 값은 1024이다. 1000은 넘지 말아야 하는 출력 제한이 아니라 반복을 계속할지 검사하는 기준이다.

# While Repetition

## 한 번도 실행되지 않는 경우와 끝나지 않는 경우

### 처음부터 조건이 거짓일 수 있다

product를 1024로 시작하면 <= 1000이 거짓이므로 본문을 한 번도 실행하지 않는다. while은 반드시 한 번 실행한 뒤 검사하는 구조가 아니다.

### 본문이 한 번도 실행되지 않음

```
product = 1024
while product <= 1000:
    product = 2 * product
```

### 상태가 변하지 않으면 조건도 계속 참일 수 있다

product를 2로 시작하고 본문에서 출력만 하면 값이 바뀌지 않는다. 조건이 계속 참이므로 출력이 반복된다. 이 코드는 그대로 실행하지 말고 원인을 먼저 설명한다.

### 종료되지 않는 잘못된 예

```
product = 2
while product <= 1000:
    print(product)
```

### 강제로 중단하는 것과 문제를 해결하는 것은 다르다

실행을 멈출 때는 터미널의 Ctrl+C나 Notebook의 Interrupt를 사용할 수 있다. 이후에는 갱신문이나 조건이 잘못된 이유를 고쳐야 한다.

### 확인할 질문

본문을 실행할 때 조건에 쓰는 값이 실제로 달라지는가? 그 변화가 종료를 가능하게 하는가?

# Practice

## 실습: 반복 횟수와 누적 합계 예상하기

### 현재 값을 더한 뒤 다음 값으로 이동한다

아래 코드에서는 i가 1부터 시작한다. total에 현재 i를 더하고 i를 1 증가시킨 뒤 다시 조건을 검사한다.

### 최종값과 사용했던 값을 구분한다

i가 4가 되면 조건이 거짓이므로 본문을 실행하지 않는다. 따라서 4는 합계에 더하지 않지만 반복 종료 후 i의 값은 4이다.

### 실행 순서를 바꾸어 생각한다

i 증가를 total 갱신보다 먼저 두면 더해지는 수가 달라진다. 같은 두 대입문이라도 본문 안의 순서에 따라 계산 결과가 달라질 수 있다.

### 먼저 결과를 예상

```
i = 1
total = 0
while i <= 3:
    total = total + i
    i = i + 1
print(total, i)
```

### 확인할 답

total은 1 + 2 + 3의 결과인 6이다.
i는 종료 조건을 만든 4이다.
출력은 6 4이다.

### 변형 결과

증가문을 먼저 실행하면 2, 3, 4를 더하여 total은 9가 된다.

# Greatest Common Divisor

## 나머지를 이용해 최대공약수 문제 줄이기

### 최대공약수는 공통 약수 중 가장 큰 수이다

48과 18의 공약수는 1, 2, 3, 6이고 최대공약수는 6이다. 모든 약수를 하나씩 찾는 대신 나머지 계산으로 더 작은 문제로 바꿀 수 있다.

### 두 수를 나머지로 갱신한다

유클리드 알고리즘은 (x, y)를 (y, x % y)로 바꾼다. 48을 18로 나눈 나머지가 12이므로 다음에는 18과 12의 최대공약수를 구한다.

### 나머지가 0이면 마지막 수가 답이다

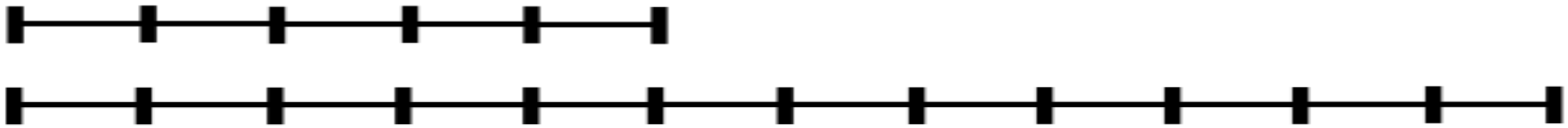
다음 쌍은 12와 6, 그다음은 6과 0이다. 두 번째 수가 0이 되면 첫 번째 수가 최대공약수이다.
이 실습은 양의 정수 두 개를 입력한다고 가정한다.

### 문제를 줄이는 순서

48, 18
18, 12
12, 6
6, 0

### 왜 같은 최대공약수인가

x = q × y + r에서 x와 y를 모두 나누는 수는 r도 나눈다. 반대로 y와 r을 나누는 수는 x도 나누므로 공통 약수들이 유지된다.



# Greatest Common Divisor

## x, y, r의 변화를 표로 따라가기

각 반복에서 r = x % y를 계산한 뒤 x, y = y, r을 실행한다. 갱신 전 y가 다음 x가 되고, 방금 계산한 나머지가 다음 y가 된다.

| 반복 시작 x | 반복 시작 y | r = x % y | 갱신 후 (x, y) |
|---|---|---|---|
| 48 | 18 | 12 | (18, 12) |
| 18 | 12 | 6 | (12, 6) |
| 12 | 6 | 0 | (6, 0) |
| 6 | 0 | 계산하지 않음 | 반복 종료 |

### 왜 0으로 나누는 오류가 생기지 않는가

y가 0이면 while y != 0이 거짓이므로 x % y를 실행하지 않는다. 종료 조건 검사를 본문보다 먼저 수행한다는 점이 안전한 실행과 연결된다.

### 마지막에 출력할 이름

반복이 끝나면 x가 최대공약수를 가리킨다. 마지막 y는 0이므로 답으로 출력할 값이 아니다.

# Greatest Common Divisor

## 해설: while로 유클리드 알고리즘 구현하기

### 조건은 “아직 나머지 계산이 필요한가”이다

while y != 0은 두 번째 수가 0이 아닌 동안 반복한다. 매번 r을 계산하고 x와 y를 갱신하므로 다음 반복은 더 작은 나머지 문제를 다룬다.

### 다중 대입은 변경 전 오른쪽 값을 사용한다

x, y = y, r에서 오른쪽의 y와 r을 먼저 읽는다. 두 값이 대응되는 이름에 대입되므로 갱신 순서로 인한 혼동을 줄일 수 있다.

### 문자열로 출력할 때 자료형을 맞춘다

마지막 출력은 문자열끼리 +로 연결한다. x는 정수이므로 str(x)를 사용한다. print("최대 공약수는", x, "입니다")처럼 여러 값으로 전달할 수도 있다.

### 예제 코드

```
x = int(input("큰 정수: "))
y = int(input("작은 정수: "))
while y != 0:
    r = x % y
    x, y = y, r
print("최대 공약수는 " + str(x) + "입니다")
```

### 48과 18을 입력한 결과

최대 공약수는 6입니다

### 코드 읽기 순서

초기값 → 조건 → 나머지 → 갱신 → 다음 조건 → 최종 출력으로 따라간다.

# Greatest Common Divisor

## 알고리즘이 멈추는 이유와 다른 입력 확인하기

### 양의 나누는 수보다 나머지가 작다

y가 양수일 때 x % y는 0 이상 y 미만이다. 갱신 후 두 번째 수가 더 작아지므로 같은 양의 값만 반복하지 않고 결국 0에 도달한다.

### 나누어떨어지는 입력은 빨리 끝난다

20과 5를 입력하면 첫 나머지가 0이다. 한 번의 본문 실행 후 (5, 0)이 되므로 최대공약수는 5이다. 반복 횟수는 입력에 따라 달라진다.

### 두 수의 순서를 바꾸어도 확인해 본다

18과 48로 시작하면 첫 나머지는 18이고 다음 쌍은 (48, 18)이 된다. 이후 계산은 앞 예제와 같아 답은 6이다. 안내에서는 큰 수부터 받지만 알고리즘은 이 순서도 처리한다.

### 다른 입력의 결과

| 입력 | 최대공약수 |
|---|---|
| 20, 5 | 5 |
| 17, 13 | 1 |
| 18, 48 | 6 |
| 36, 36 | 36 |

### 입력 범위는 유지한다

지금은 양의 정수 예제로 원리를 익힌다. 0이나 음수를 일반적으로 허용하려면 입력 조건과 결과의 의미를 따로 정해야 한다.

# For Repetition

## for: 다음 원소를 하나씩 가져와 실행하기

### 반복할 대상에서 원소를 가져온다

for는 대상의 원소를 순서대로 이름에 대입하고 본문을 실행한다. 다음 원소가 없으면 반복을 끝내고 반복문 뒤의 문장으로 이동한다.

### 반복 변수를 따로 증가시키지 않는다

for counter in range(3)에서는 counter가 순서대로 0, 1, 2를 받는다. while 예제처럼 counter = counter + 1을 본문에 적을 필요가 없다.

### 정해진 횟수의 반복만 가능한 것은 아니다

문자열의 문자처럼 순회 가능한 원소도 사용할 수 있다. 이 단원에서는 정수 범위를 다루는 range와 함께 사용하는 형태를 중심으로 배운다.

### range를 사용하는 예

```
for counter in range(3):
    print(counter)
print("Done")
```

### 실행 결과

```
0
1
2
Done
```

### 문자열을 순회하는 다른 예

```
for letter in "AI":
    print(letter)
```

# For Repetition

## range의 끝값은 포함되지 않는다

### range(stop)은 0부터 시작한다

range(5)는 0, 1, 2, 3, 4를 차례로 제공한다. 5는 마지막으로 포함되는 값이 아니라 멈출 경계이다. 따라서 본문은 다섯 번 실행된다.

### range(start, stop)은 시작값을 지정한다

range(1, 5)는 1부터 시작하되 5는 제외하므로 1, 2, 3, 4를 제공한다. 1부터 5까지 필요하면 stop을 6으로 지정한다.

### Python 3의 range는 리스트와 구분한다

range는 정수 범위를 표현하는 불변 시퀀스 객체이다. 모든 값을 담은 리스트를 미리 만드는 함수라고 이해하지 않는다. for에서 필요한 값을 순서대로 사용할 수 있다.

### 정수 범위 비교

| 표현식 | 제공되는 값 |
|---|---|
| range(5) | 0, 1, 2, 3, 4 |
| range(1, 5) | 1, 2, 3, 4 |
| range(1, 6) | 1, 2, 3, 4, 5 |
| range(0) | 없음 |

### 확인할 질문

시작값, 마지막 포함 값, 반복 횟수를 서로 다른 정보로 구분한다.

# For Repetition

## range(start, stop, step)으로 간격 지정하기

### 양의 step은 값을 증가시킨다

range(1, 8, 2)는 1에서 시작해 2씩 증가한다. 1, 3, 5, 7을 제공하고 다음 값 9는 경계를 넘으므로 사용하지 않는다.

### 음의 step은 값을 감소시킨다

range(5, 0, -2)는 5, 3, 1을 제공한다. 감소할 때도 stop은 포함되지 않는다. 시작값과 끝값의 방향에 맞는 step을 선택해야 한다.

### 잘못된 방향과 0을 구분한다

range(5, 0, 1)은 오류가 아니라 빈 범위이다. 반면 step을 0으로 정하면 값이 진행할 수 없으므로 ValueError가 발생한다.

### 간격과 방향

| 표현식 | 제공되는 값 |
|---|---|
| range(1, 8, 2) | 1, 3, 5, 7 |
| range(5, 0, -2) | 5, 3, 1 |
| range(5, 0) | 없음 |
| range(1, 5, 0) | ValueError |

### 직접 읽어 보기

range(10, 0, -3)은 어떤 값을 제공하는가?
10, 7, 4, 1이다.

# Practice

## 실습: for로 팩토리얼 계산하기

### 팩토리얼은 1부터 n까지의 곱이다

양의 정수 n에 대해 n! = 1 × 2 × … × n이다.
예를 들어 4!은 1 × 2 × 3 × 4 = 24이고
10!은 3628800이다.

### 누적 곱을 저장할 이름이 필요하다

fact에 지금까지의 곱을 저장한다. 다음 정수 i를 받을 때마다 기존 fact에 i를 곱하고 결과를 다시 fact에 대입한다.

### 초기값과 반복 범위를 정한다

곱셈의 누적값을 0으로 시작하면 모든 결과가 0이 된다. 어떤 초기값이 필요한지, n을 마지막에 포함하려면 range의 끝값을 무엇으로 해야 하는지 먼저 생각한다.

### 완성할 코드

```
n = int(input("정수: "))
fact = _______
for i in range(_______, _______):
    fact = _____________
print(n, "!은", fact, "이다.")
```

### 시험할 입력

4를 입력하면 24이다.
10을 입력하면 3628800이다.
실습은 0 이상의 정수를 입력한다고 가정한다.

# Practice

## 해설: 누적 곱의 초기값은 1이다

### 초기값 1은 첫 곱셈을 방해하지 않는다

fact = 1에서 시작하여 i를 하나씩 곱한다.
첫 i가 1이면 결과는 1이고, 다음 i가 2이면 2,
다음이 3이면 6이 된다.

### n을 포함하려면 끝값은 n + 1이다

range(1, n + 1)은 1부터 n까지의 정수를 제공한다.
range(1, n)을 사용하면 마지막 n을 곱하지 않아
목표와 다른 값을 만든다.

### 마지막 출력은 반복문 밖에 둔다

print()가 본문 밖에 있으므로 전체 곱셈이 끝난 뒤
한 번 출력한다. 본문 안에 두면 각 반복의 중간
결과를 출력하게 된다.

### 완성 코드

```
n = int(input("정수: "))
fact = 1
for i in range(1, n + 1):
    fact = fact * i
print(n, "!은", fact, "이다.")
```

### 10을 입력한 결과

10 !은 3628800 이다.

### 합계 누적과 비교

덧셈의 누적값은 보통 0에서 시작하고
곱셈의 누적값은 1에서 시작한다. 사용할
연산에 맞는 초기값을 정한다.

# For Repetition

## 팩토리얼의 중간 상태와 0! 확인하기

아래 표는 n = 4인 변형 예제이다. fact는 매번 현재 i까지 곱한 결과를 유지한다.
갱신 전과 후를 나누면 누적 계산을 정확히 읽을 수 있다.

| i | 갱신 전 fact | 계산 | 갱신 후 fact |
|---|---|---|---|
| 1 | 1 | 1 × 1 | 1 |
| 2 | 1 | 1 × 2 | 2 |
| 3 | 2 | 2 × 3 | 6 |
| 4 | 6 | 6 × 4 | 24 |

# For Repetition

## 실행은 되지만 계산은 틀릴 수 있다

### 잘못된 초기값은 모든 반복에 영향을 준다

fact를 0으로 시작하면 어떤 정수를 곱해도 계속 0이다. 오류 메시지 없이 실행되므로 출력값을 예상 결과와 비교해야 찾을 수 있다.

### 끝값을 잘못 적으면 마지막 항이 빠진다

n = 4에서 range(1, 4)는 1, 2, 3만 제공한다. 따라서 계산 결과는 6이며 4!의 24가 아니다. 마지막 포함 값과 stop을 구분한다.

### 경계 입력으로 확인한다

0, 1, 4처럼 다른 크기의 입력을 시험한다. 반복 횟수, 초기값, 갱신문, 출력 위치를 각각 확인하면 원인을 좁힐 수 있다.

### 초기값이 잘못된 예

```
n = 4
fact = 0
for i in range(1, n + 1):
    fact = fact * i
```

### 결과

fact는 0이다.

### 끝값이 잘못된 예

```
fact = 1
for i in range(1, n):
    fact = fact * i
```

### 결과

fact는 6이다. 마지막 4가 빠졌다.

# Loop Control

## break: 가장 가까운 반복문을 종료하기

### break를 실행하면 본문을 더 진행하지 않는다

x가 5일 때 break를 만나므로 같은 반복의 print(x)는 실행되지 않는다. 이후 6부터 10까지의 원소도 처리하지 않는다.

### 종료하는 대상은 반복문이다

break가 전체 프로그램을 종료하는 것은 아니다. 반복문 바로 뒤에 있는 문장으로 이동하므로 마지막 출력은 실행된다.

### 마지막 출력값과 반복 변수의 값을 구분한다

본문에서 마지막으로 출력한 값은 4이지만 break를 실행할 때 x는 5이다. 반복을 멈추게 한 값과 출력된 마지막 값은 같지 않을 수 있다.

### 예제 코드

```
for x in range(1, 11):
    if x == 5:
        break
    print(x)
print("반복 종료 시 x =", x)
```

### 실행 결과

```
1
2
3
4
반복 종료 시 x = 5
```

# Loop Control

## continue: 현재 반복의 남은 부분만 건너뛰기

### 다음 반복으로 진행한다

x가 5이면 continue를 실행하여 그 아래의 print(x)를 건너뛴다. 반복 자체는 끝나지 않으므로 다음 값 6으로 진행한다.

### 건너뛰는 것은 현재 본문의 남은 문장이다

x가 1, 2, 3, 4일 때는 출력하고 5일 때만 출력하지 않는다. 이후 6, 7, 8, 9, 10도 출력한다. 5가 범위에서 제거된 것이 아니라 출력문을 실행하지 않은 것이다.

### 예제 코드

```
for x in range(1, 11):
    if x == 5:
        continue
    print(x)
print("반복 종료")
```

### 출력되는 값

1, 2, 3, 4, 6, 7, 8, 9, 10이 각각 한 줄씩 출력된다. 마지막에는 “반복 종료”가 출력된다.

### 종료 시 x

반복이 10까지 진행되므로 마지막 x는 10이다.

# Loop Control

## 같은 조건에서 break와 continue 비교하기

두 예제 모두 range(1, 11)에서 x == 5를 검사한다. 차이는 조건이 참일 때 반복문 밖으로 나오는지, 다음 원소로 진행하는지에 있다.

| 확인할 내용 | break | continue |
|---|---|---|
| x = 5의 print(x) | 실행하지 않음 | 실행하지 않음 |
| x = 6 이후 처리 | 수행하지 않음 | 수행함 |
| 본문에서 출력한 값 | 1, 2, 3, 4 | 1, 2, 3, 4, 6, 7, 8, 9, 10 |
| 반복 종료 후 x | 5 | 10 |
| 반복문 밖의 문장 | 계속 실행 | 계속 실행 |

### 사용 목적을 먼저 정한다

필요한 대상을 찾았으니 더 볼 필요가 없으면 break를 생각할 수 있다.
특정 대상만 처리하지 않고 나머지를 계속 볼 때는 continue가 적합하다.

# Practice

## 종합 실습: 조건과 반복을 함께 추적하기

### 반복마다 실행하는 경로를 확인한다

i는 1부터 7까지 순서대로 들어올 수 있다. 다만 조건문에 의해 어떤 반복은 건너뛰고 어떤 값에서는 반복이 종료될 수 있다.

### 누적에 포함되는 값을 따로 적는다

total 갱신문에 도달한 i만 합계에 포함된다. range에 들어 있는 모든 값을 더하는 코드라고 생각하지 않는다.

### 두 가지 결과를 예상한다

마지막 total과 i를 각각 적는다. 실행 전에 포함된 값의 목록을 만들고, 3과 6에서 다음에 실행할 줄이 어디인지 설명한다.

### 실습 코드

```
total = 0
for i in range(1, 8):
    if i == 3:
        continue
    if i == 6:
        break
    total = total + i
print(total, i)
```

### 질문

어떤 값들이 합계에 포함되는가?
3은 왜 제외되는가?
6과 7은 왜 더하지 않는가?

# Practice

## 해설: 더해진 값과 종료한 값을 구분하기

1과 2는 누적하고 3은 continue로 건너뛴다. 4와 5를 더한 뒤 6에서 break를 만나므로 누적하지 않고 반복문 밖으로 이동한다.

| i | 선택한 동작 | 동작 후 total |
|---|---|---|
| 1 | 누적 | 1 |
| 2 | 누적 | 3 |
| 3 | continue | 3 |
| 4 | 누적 | 7 |
| 5 | 누적 | 12 |
| 6 | break | 12 |

**최종 출력은 12 6이다**

합계는 1 + 2 + 4 + 5 = 12이다. i는 종료를 일으킨 6이며, 7은 처리하지 않는다. 마지막으로 더한 값은 5지만 반복 종료 후 i는 6이라는 점을 구분한다.

# Chapter Overview

## 제어 구조는 다음에 실행할 문장을 결정한다

### 선택 구조

if는 조건이 참일 때 본문을 실행한다. if/elif/else는 처음 참인 경로 하나를 선택하므로 조건의 범위와 순서가 중요하다. 경계값을 직접 넣어 확인한다.

### 반복 구조

while은 초기값·조건·갱신·종료의 관계를 확인한다. for와 range는 시작값·끝 경계·간격을 구분한다. 최대공약수와 팩토리얼은 이 원리를 적용한 예제이다.

### 결과 검증

break는 반복을 종료하고 continue는 현재 반복의 남은 부분을 건너뛴다. 최종 값만 외우지 말고, 각 문장이 실행될 때의 값과 다음 실행 위치를 설명할 수 있어야 한다.

# CBTL 혁신수업 운영 설문 조사 (교수학습센터)

- **운영:** 사전 / 사후 각 1회 설문 조사 진행 예정 (사후는 추후 공지)
- **링크**: https://forms.gle/ogzcp5VVpfWdSfTp7
- **기타:** 해당 설문 조사 내용은 평가에 반영되지 않고, 교수학습센터의 자료로 활용